ОТЧЕТ

Тема: «Моделирование движения на перекрестке дорог»

Фролов И., Морозова А.

2026 г.

Содержание

1 Уточненная постановка задачи для назначенного варианта задания 2
1.1 Моделируемые объекты и топология перекрестка 2
1.2 Стохастические факторы и законы движения 2
1.3 Режимы управления светофорными объектами 3
1.4 Изменяемые параметры моделирования 4
1.5 Выходные аналитические показатели 4

2 Диаграмма классов программной системы 5
2.1 Взаимосвязи между компонентами архитектуры 5

3 Текстовые спецификации основных классов системы 6
3.1 Модуль моделируемых сущностей (`src/core/entities/`) 6
3.2 Модуль физики и моделирования (`src/core/simulation/`) 6
3.3 Модуль управления светофорными объектами (`src/core/traffic_control/`) 7
3.4 Модуль аналитики (`src/core/analytics/`) 7
3.5 Модуль графического интерфейса (`src/ui/`) 7

4 Использованные инструментальные средства 8

5 Описание файловой структуры программной системы 8

6 Краткая характеристика пользовательского интерфейса 9
6.1 Центральная графическая область (`SimulationView` / `SimulationScene`) . . 9
6.2 Левая док-панель («Управление и Аналитика») 10
6.3 Правая док-панель («Параметры») 11

7 Результаты проведенных вычислительных экспериментов 11
7.1 Графики сравнения и выводы по экспериментам 12

8 Подготовка полного GIT-репозитория проекта на GitHub 13

---

1 Уточненная постановка задачи для назначенного варианта задания

Разрабатываемая программная система предназначена для имитационного моделирования и визуализации транспортных и пешеходных потоков на регулируемом перекрестке двух автомобильных дорог с целью исследования закономерностей возникновения и рассасывания дорожных заторов в зависимости от интенсивности движения и алгоритмов светофорного регулирования.

1.1 Моделируемые объекты и топология перекрестка

- **Перекресток дорог**: пересечение двух многополосных автомобильных дорог в декартовой системе координат с центром в точке $(0, 0)$. Реализована поддержка трех топологических конфигураций:
  1. $2 + 2$: перекресток двух четырехполосных дорог (по 2 полосы в каждом направлении, ширина проезжей части одного направления 7.0 м).
  2. $2 + 3$ (Асимметрия): двухполосные подходы по оси «Север–Юг» (7.0 м) и трехполосные подходы по оси «Восток–Запад» (10.5 м).
  3. $3 + 3$: перекресток двух шестиполосных дорог (по 3 полосы в каждом направлении, 10.5 м).
- **Геометрия полос**: ширина одной полосы движения $w_{\text{lane}} = 3.5$ м. Длина подходов дорог от центра перекрестка составляет не менее 100 м.
- **Стоп-линии**: расположены на удалении 4.0 м от физических границ зоны пересечения проезжих частей.
- **Пешеходные переходы (зебры)**: ширина переходов 3.0 м, они расположены между стоп-линиями и границей пересечения дорог (с зазором 0.5 м от стоп-линии).
- **Транспортные средства (автомобили)**: габаритные размеры 4.5×1.8 м. Модель включает визуализацию лобового и заднего стекол, боковых зеркал заднего вида, стоп-сигналов и динамических указателей поворота.
- **Пешеходы**: точечные объекты радиусом $R = 0.6$ м, двигающиеся вдоль тротуаров и пересекающие перекресток по выделенным переходам.

1.2 Стохастические факторы и законы движения

- **Появление автомобилей**: ТС генерируются на внешних границах подходов (100 м от центра) согласно пуассоновскому процессу с заданной интенсивностью P (авт/ч) для каждого направления. Временные интервалы между последовательными появлениями распределены по экспоненциальному закону:

$$\Delta t_{\text{arrival}} = -\frac{3600}{P} \ln(U), \quad U \sim \text{Uniform}(0, 1) \tag{1}$$

- **Скорость ТС**: начальная скорость v_0 определяется случайной величиной из диапазона $[V_{\text{min}}, V_{\text{max}}]$ (от 20 до 140 км/ч).
- **Маршрут движения**: маневр определяется стохастически при появлении ТС на границе мира: «прямо» (60%), «налево» (20%), «направо» (20%).

- **Выбор полосы и маневрирование**: перестроение перед стоп-линией осуществляется строго по ПДД (поворот налево выполняется с крайней левой полосы, поворот направо — с крайней правой).

- **Следование за лидером (Intelligent Driver Model, IDM)**: продольное ускорение a каждого автомобиля рассчитывается непрерывно по формуле:

$$a = a_{\text{max}} \left[1 - \left(\frac{v}{v_0} \right)^4 - \left(\frac{s^*(v, \Delta v)}{s} \right)^2 \right] \tag{2}$$

  где динамическая безопасная дистанция $s^*(v, \Delta v)$ равна:

$$s^*(v, \Delta v) = s_0 + vT_h + \frac{v\Delta v}{2\sqrt{a_{\text{max}} b}} \tag{3}$$

  Базовые параметры модели: максимальное ускорение $a_{\text{max}} = 1.5$ м/с^2, комфортное замедление $b = 2.0$ м/с^2, минимальная дистанция в заторе $s_0 = 2.0$ м, безопасный временной интервал $T_h = 1.2$ с.

- **Реакция на светофор и стоп-линию**: при запрещающем сигнале стоп-линия интерпретируется моделью как неподвижное фиктивное транспортное средство ($v_{\text{lead}} = 0$). Реакция начинается в пределах дальности видимости D. Учитывается дилемма желтого сигнала: если необходимое замедление превышает критический порог экстренного торможения $b_{\text{crit}} = 4.5$ м/с^2, автомобиль завершает проезд перекрестка.

- **Повороты и приоритеты**: маневры поворота выполняются по дугам окружностей с безопасным ограничением скорости $v_{\text{turn}} = \sqrt{3R} \cdot 3.6$ км/ч ($v_{\text{turn}} \in [15, 40]$ км/ч). Реализована поддержка режима «просачивания» (ВСТР): ТС выезжает к центру перекрестка и уступает дорогу встречному потоку и пешеходам. При отключении режима маневр осуществляется по отдельной стрелке левой секции светофора.

- **Пешеходное движение**: пуассоновский поток с интенсивностью P_{ped} (50–1000 пеш/ч). Скорость пешеходов равномерно распределена в интервале $[1.0, 1.5]$ м/с. Поддерживается движение параллельно попутному транспорту либо в выделенной фазе (All-Red).

- Вероятность столкновений исключена за счет поддержания физического барьера безопасности ($s \leq 2.0$ м) и алгоритмического бесконфликтного разъезда.

1.3 Режимы управления светофорными объектами

1. **Статический режим (`StaticController`)**: жесткая циклограмма с настраиваемыми интервалами зеленого (Z) и красного (K) сигналов в рамках общего цикла регулирования T. Предусмотрены промежуточные такты желтого (3.0 с) и красно-желтого (1.5 с) сигналов.

2. **Адаптивный / Динамический режим (`DynamicController`)**: гибкое регулирование длительности фазы на основе виртуальных детекторов очередей ($v < 2.0$ м/с) в зоне видимости D. Реализованы минимальное время зеленого $Z_{\text{min}} = \max(12, 4 + 1.8 \cdot N_{\text{queue}})$ с, максимальный лимит $Z_{\text{max}} = 45$ с, режим Rest-in-Green и ожидание очистки центра перекрестка перед сменой фазы.

1.4 Изменяемые параметры моделирования

- Режим работы светофора: статический / адаптивный.
- Топология перекрестка: $2+2$, $2+3$ (Асимметрия), $3+3$.
- Разрешение просачивания налево во встречный поток (`permitLeftTurnFilter`).
- Режим параллельного пропуска пешеходов с автотранспортом.
- Интенсивности транспортных потоков P для 4 подходов: Север, Юг, Восток, Запад (100–1000 авт/ч).
- Длительности зеленого Z и красного K сигналов для каждого подхода (5–180 с).
- Общий цикл светофорного регулирования T (30–180 с).
- Дистанция видимости светофора D (20–150 м).
- Длительность пешеходной зеленой фазы Z_{ped} (5–60 с) и поток пешеходов P_{ped} (50–1000 пеш/ч).
- Границы скоростного режима ТС: $[V_{\text{min}}, V_{\text{max}}]$ (20–140 км/ч).
- Шаг дискретизации моделирования по времени: Δt (0.033 с / 0.05 с).

1.5 Выходные аналитические показатели

- Среднее время задержки автомобилей в очереди: $T_{\text{avg_wait}} = \frac{1}{N}\sum_{i=1}^{N} t_{\text{wait},i}$ (с).
- Текущий размер очереди (количество остановившихся автомобилей, $v < 0.5$ м/с).
- Суммарное количество автомобилей, успешно покинувших зону перекрестка.
- Динамический график изменения длины очереди в реальном времени (скользящее окно в 60 с).
- Экспорт массива статистических данных симуляции в формат CSV.

2 Диаграмма классов программной системы

Архитектура системы построена на базе шаблона проектирования Model-View-Presenter (MVP). Математическое и физическое ядро инкапсулировано в библиотеке `TrafficCore` и изолировано от графической подсистемы Qt Widgets.

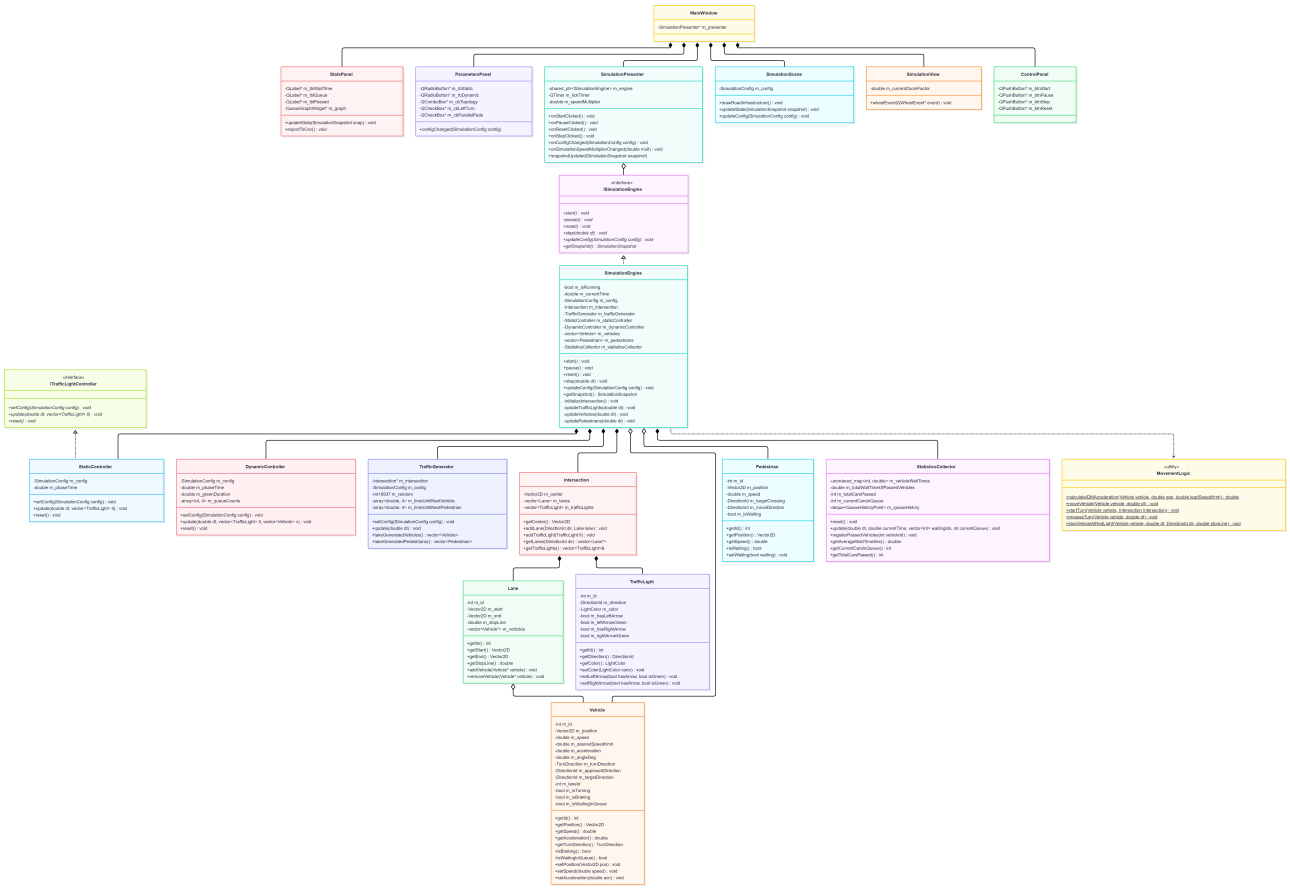


Рис. 1: Диаграмма классов программного комплекса моделирования перекрестка

2.1 Взаимосвязи между компонентами архитектуры

- Фасадный интерфейс `ISimulationEngine` определяет методы управления симуляцией (`start`, `pause`, `step`, `reset`) и получения среза данных (`getSnapshot`).
- Класс `SimulationEngine` реализует интерфейс модели и объединяет модули инфраструктуры (`Intersection`), генерации (`TrafficGenerator`), управления светофорами (`StaticController`, `DynamicController`), физики движения (`MovementLogic`) и статистики (`StatisticsCollector`).
- Презентер `SimulationPresenter` принимает сигналы таймера с частотой 30 кадров/с, выполняет квант времени Δt в ядре и передает снимок `SimulationSnapshot` в представление (`SimulationView`, `SimulationScene`, `StatsPanel`).

3 Текстовые спецификации основных классов системы

3.1 Модуль моделируемых сущностей (`src/core/entities/`)

- **`Vehicle`**:
  - *Поля*: `m_id` (уникальный номер ТС), `m_position` (вектор координат $\vec{r}$), `m_speed` (текущая скорость), `m_desiredSpeedKmh` (желаемая скорость), `m_acceleration` (ускорение a), `m_angleDeg` (угол курса), `m_turnDirection` (маневр: прямо, влево, вправо), `m_approachDirection` (подход прибытия), `m_targetDirection` (целевое направление), `m_laneId` (номер полосы), `m_isTurning` (флаг выполнения поворота), `m_turnProgress` (прогресс прохождения дуги $[0, 1]$), `m_turnCenter`, `m_turnRadius`, `m_isBraking` (флаг торможения), `m_isWaitingInQueue` (флаг ожидания в заторе).
  - *Методы*: геттеры и сеттеры кинематического состояния, расчет углов траектории, управление флагами очередей и торможения.
- **`Pedestrian`**:
  - *Поля*: `m_id`, `m_position`, `m_speed` (1.0–1.5 м/с), `m_targetCrossing` (целевой переход), `m_moveDirection` (направление движения), `m_isWaiting` (флаг ожидания разрешающего сигнала на границе тротуара).
  - *Методы*: чтение и модификация координат, скорости, направления и статуса ожидания.
- **`Lane`**:
  - *Поля*: `m_id`, `m_start`, `m_end` (геометрические координаты оси полосы), `m_stopLine` (координата стоп-линии), `m_vehicles` (список указателей на ТС, находящиеся на полосе).
  - *Методы*: `addVehicle(Vehicle*)`, `removeVehicle(Vehicle*)`, `getVehicles()`, `getStopLine()`.
- **`TrafficLight`**:
  - *Поля*: `m_id`, `m_direction`, `m_color` (сигналы Red, Yellow, Green, RedYellow), `m_hasLeftArrow`, `m_leftArrowGreen`, `m_allowsParallelPeds`, `m_parallelPedsGreen`.
  - *Методы*: установка и считывание основных сигналов и дополнительных стрелочных секций.

3.2 Модуль физики и моделирования (`src/core/simulation/`)

- **`Intersection`**:
  - *Поля*: `m_center`, `m_lanes` (вектор полос движения), `m_lanesByDirection` (индексация полос по сторонам света), `m_trafficLights` (вектор светофоров).
  - *Методы*: инициализация геометрии перекрестка в зависимости от выбранной топологии, выборка полос по направлению движения.
- **`MovementLogic`**:
  - *Методы (статические)*:
    - `calculateIDMAcceleration(...)`: расчет продольного ускорения по формуле модели IDM с учетом дистанции и скорости лидера.

- `moveVehicle(Vehicle&, double dt)`: численное интегрирование скорости и координат прямолинейного движения.
- `startTurn(…)`, `processTurn(…)`: расчет криволинейной кинематики движения по дуге окружности.
- `stopVehicleAtRedLight(…)`: остановка автомобиля перед стоп-линией на запрещающий сигнал.

- **`TrafficGenerator`**:
  - *Поля*: указатель на `Intersection`, генератор псевдослучайных чисел `std::mt19937`, массив таймеров появления ТС и пешеходов `m_timeUntilNextVehicle`, `m_timeUntilNext`
  - *Методы*: `update(dt)`, генерация интервалов по экспоненциальному закону, генерация скоростей, выбор полосы спавна согласно правилам ПДД.
- **`SimulationEngine`**:
  - *Поля*: флаг `m_isRunning`, физическое время `m_currentTime`, конфигурация `SimulationCon` экземпляры модулей ядра и контейнеры сущностей `m_vehicles`, `m_pedestrians`.
  - *Методы*: реализация интерфейса `ISimulationEngine`, диспетчеризация движения на шаге `step(dt)`, спавн в хвост пробки при выходе за пределы сцены, разрешение конфликтов встречного разъезда при левом повороте, удаление ТС, покинувших область симуляции (> 110 м).

3.3 Модуль управления светофорными объектами (`src/core/traffic_control/`)

- **`ITrafficLightController`**: интерфейс с виртуальными методами `setConfig`, `update` и `reset`.
- **`StaticController`**: реализует жесткую циклограмму переключения фаз с учетом промежуточных тактов (желтый 3.0 с, красно-желтый 1.5 с, пешеходная фаза).
- **`DynamicController`**: осуществляет опрос виртуальных детекторов очередей, рассчитывает задержки и производит адаптивное продление зеленой фазы либо досрочное переключение при рассасывании затора.

3.4 Модуль аналитики (`src/core/analytics/`)

- **`StatisticsCollector`**: ведет учет индивидуального времени задержки каждого ТС (`m_vehicleWaitTimes`), подсчитывает суммарное число выехавших автомобилей и сохраняет кольцевой буфер истории очередей за последние 60 секунд.

3.5 Модуль графического интерфейса (`src/ui/`)

- **`SimulationPresenter`**: управляющий класс, связывающий ядро модели и графические компоненты по таймеру 30 FPS.
- **`SimulationView` / `SimulationScene`**: холст на базе Qt Graphics Framework с поддержкой масштабирования колесиком мыши, панорамирования и отрисовки дорожной разметки, стоп-линий, зебр, светофоров и автомобилей.
- **`ParametersPanel`**: док-панель параметров перекрестка с дебаунс-таймером обновления настроек (50 мс).
- **`StatsPanel`**: док-панель вывода задержек, очередей, построения динамического графика и экспорта данных в CSV.

4 Использованные инструментальные средства

1. **Язык программирования**: C++ (стандарт C++17). Использованы возможности стандартной библиотеки STL: алгоритмы генерации псевдослучайных величин (<random>), контейнеры (std::vector, std::deque, std::unordered_map), умные указатели.
2. **Графический фреймворк**: Qt 6 (модули Qt6::Core, Qt6::Gui, Qt6::Widgets). Применены механизмы метаобъектной системы, сигналов и слотов, графическая сцена QGraphicsView/QGraphicsScene и стилизация QSS.
3. **Интегрированная среда разработки (IDE)**: Qt Creator.
4. **Система автоматизации сборки**: CMake (версия ≥ 3.16) с генератором сборочных сценариев Ninja.
5. **Среда воспроизводимости зависимостей**: Nix / Nix-shell (shell.nix), обеспечивающая автоматическую изоляцию компилятора GCC, библиотек Qt6 и сборочных утилит.
6. **Система контроля версий**: Git, удаленный хостинг исходного кода GitHub.

5 Описание файловой структуры программной системы

```
TrafficSimulation/
|-- CMakeLists.txt                  # Корневой сборочный сценарий CMake
|-- shell.nix                       # Окружение зависимостей для Nix-shell
|-- .gitignore                      # Правила исключения временных файлов
|-- resources/                      # Ресурсы приложения
|   |-- resources.qrc               # XML-дескриптор ресурсов Qt
|   |-- icons/                      # SVG-иконки интерфейса (play, pause, step)
|   -- styles/
|       -- style.qss               # Таблица стилей интерфейса (QSS)
|-- src/                            # Исходный код приложения
|   |-- main.cpp                    # Точка входа в программу
|   |-- common/                     # Общие типы данных и контракты
|   |   |-- Types.h                 # Перечисления: фазы, цвета, направления
|   |   |-- Vector2D.h              # Математический 2D-вектор
|   |   |-- SimulationConfig.h      # Структура параметров моделирования
|   |   |-- ISimulationEngine.h     # Интерфейс движка и снимок Snapshot
|   |   -- MockSimulationEngine.h  # Заглушка для тестирования UI
|   |-- core/                       # Ядро модели (библиотека TrafficCore)
|   |   |-- entities/               # Моделируемые сущности
|   |   |   |-- Vehicle.h / .cpp    # Класс транспортного средства
|   |   |   |-- Pedestrian.h / .cpp # Класс пешехода
|   |   |   |-- TrafficLight.h/.cpp # Класс светофорного объекта
|   |   |   -- Lane.h / .cpp       # Класс полосы движения со стоп-линией
|   |   |-- simulation/             # Логика моделирования
|   |   |   |-- Intersection.h/.cpp # Топология перекрестка
|   |   |   |-- TrafficGenerator.h/c# Стохастический генератор потоков
|   |   |   |-- MovementLogic.h/.cpp# Физика IDM, маневры поворотов
|   |   |   -- SimulationEngine.h/c# Цикл квантования времени (dt)
|   |   |-- traffic_control/        # Алгоритмы управления светофорами
|   |   |   |-- ITrafficLightController.h # Интерфейс контроллера
|   |   |   |-- StaticController.h/c# Контроллер жестких фаз
|   |   |   -- DynamicController.h/# Адаптивный контроллер очередей
|   |   -- analytics/              # Аналитический учет
|   |       -- StatisticsCollector.h/.cpp # Подсчет задержек и очередей
|   -- ui/                         # Пользовательский интерфейс (Qt Widgets)
|       |-- mainwindow.h / .cpp/.ui # Главное окно приложения
|       |-- canvas/                 # Модуль графической сцены
```

```
|       |   |-- SimulationView.h/.cp# Представление QGraphicsView
|       |   |-- SimulationScene.h/cp# Сцена с отрисовкой перекрестка
|       |   -- items/              # Графические элементы сцены
|       |-- panels/                 # Док-панели интерфейса
|       |   |-- ControlPanel.h/.cpp # Управление (пуск, пауза, шаг)
|       |   |-- ParametersPanel.h/cp# Панель параметров симуляции
|       |   -- StatsPanel.h / .cpp # Метрики, график очереди и CSV
|       -- presenter/              # Слой связи MVP
|           -- SimulationPresenter.h/.cpp # Связывание ядра и интерфейса
-- tools/                         # Автоматизация экспериментов
    -- experiment_runner.cpp       # Консольный раннер серий прогонов
```

6 Краткая характеристика пользовательского интерфейса

Графический интерфейс системы разработан на базе фреймворка Qt 6 Widgets в соответствии с архитектурным шаблоном Model-View-Presenter (MVP). Оформление выполнено в современной высококонтрастной темной теме посредством таблиц стилей Qt Style Sheets (`style.qss`) с акцентной синей палитрой (#3D5AFE). Компоновка главного окна включает три ключевые зоны: центральную интерактивную графическую область моделирования и две функциональные док-панели (`QDockWidget`), поддерживающие перемещение и открепление.

6.1 Центральная графическая область (`SimulationView` / `SimulationScene`)

Графическая подсистема реализована на базе специализированного представления `SimulationVie` (наследник `QGraphicsView`) и сцены `SimulationScene` (200×200 м в метрической системе координат):

- **Интерактивная навигация**: реализовано плавное масштабирование сцены колесиком мыши (с коэффициентом от $0.5\times$ до $3.0\times$ с центрированием под курсором) и свободное двумерное панорамирование перетаскиванием (режим `ScrollHandDrag`).
- **Отрисовка дорожной инфраструктуры**:
  - Проезжая часть темно-серого асфальтового оттенка (#2B2D42) с двойной сплошной осевой линией желтого цвета (#F4D03F), белыми прерывистыми разделителями полос и сплошными стоп-линиями толщиной 0.15 м.
  - Пешеходные тротуары со скругленными границами радиусом $R = 3.5$ м на углах перекрестка.
  - Четыре пешеходных перехода («зебры») шириной 3.0 м с процедурной генерацией контрастных белых полос с шагом 1.0 м.
- **Визуализация светофорных объектов**:
  - *Автомобильные светофоры* (`TrafficLightGraphicsItem`): трехсекционные вертикальные стойки с корпусом графитового цвета, поддерживающие динамическую индикацию основных сигналов (Red, Yellow, Green, RedYellow) и дополнительной секции стрелки левого поворота.
  - *Пешеходные светофоры* (`PedestrianTrafficLightGraphicsItem`): комплексные Г-образные опоры на всех четырех углах перекрестка, каждая из которых несет по два независимых двухсекционных излучателя для раздельной индикации переходов по направлениям «Север–Юг» и «Восток–Запад».
- **Модели транспортных средств (`VehicleGraphicsItem`)**:

- Масштабная геометрия кузова 4.5×1.8 м со скруглением углов, зеркалами заднего вида, лобовым и задним остеклением со световыми бликами.
- Цветовое кодирование маневров: синий кузов (#2196F3) — движение прямо, оранжевый (#FF9800) — поворот налево, фиолетовый (#9C27B0) — поворот направо.
- Световая сигнализация: передние фары, габаритные огни, динамические стоп-сигналы ярко-красного цвета (#FF1744), активирующиеся при торможении ($a < -0.5$ м/с^2), и мигающие желтые указатели поворота (период 0.66 с, скважность 50%).

- **Модели пешеходов (`PedestrianGraphicsItem`)**: векторные диски радиусом 0.6 м с белой окантовкой. Двухцветная статусная индикация: ярко-зеленый цвет (#00E676) сигнализирует о движении по переходу, а янтарно-желтый (#FFA000) — об ожидании разрешающего сигнала на тротуаре.

6.2 Левая док-панель («Управление и Аналитика»)

Панель объединяет управление воспроизведением, глобальные параметры и блок вывода аналитических метрик:

1. **Блок управления симуляцией (`ControlPanel`)**:
   - Сегментированная панель кнопок быстрого доступа со стилизованными векторными иконками: «Старт» (`play.svg`), «Пауза» (`pause.svg`), покадровый «Шаг» (`step_forward.svg`) и «Сброс» (`stop.svg`).
   - Цифровой регулятор скорости моделирования (`QDoubleSpinBox`) в диапазоне от $0.1\times$ до $5.0\times$ (с шагом $0.1\times$, значение по умолчанию $1.00\times$).
2. **Группа «Глобальные параметры»**:
   - Дублированное управление через спаренные ползунки (`QSlider`) и поля ввода (`QSpinBox`):
     - «Общий цикл (с)»: суммарный цикл светофорного регулирования $T \in [30, 180]$ с (по умолчанию 90 с).
     - «Видимость (м)»: дальность обнаружения светофора и заторов $D \in [20, 150]$ м (по умолчанию 60 м).
     - «З-ый пеш-в (с)»: длительность зеленого пешеходного сигнала $Z_{\text{ped}} \in [5, 60]$ с (по умолчанию 15 с).
     - «Поток пеш-в»: общая интенсивность пешеходов $P_{\text{ped}} \in [50, 1000]$ пеш/ч (по умолчанию 300).
     - «Ск-ть min (км/ч)» и «Ск-ть max (км/ч)»: допустимый диапазон скоростей ТС от 20 до 140 км/ч со встроенной перекрестной валидацией ($V_{\text{min}} \leq V_{\text{max}}$).
   - Автоматическая деактивация недоступных настроек (например, общего цикла T и времени пешеходного зеленого Z_{ped} при переключении в адаптивный режим или при совмещенной пешеходной фазе).
3. **Блок оперативной аналитики (`StatsPanel`)**:
   - Цифровые табло реального времени: «Средняя задержка» (с точностью до 0.1 с), «Очередь (авто)» (число остановившихся ТС, $v < 0.5$ м/с) и «Проехало машин» (суммарный счетчик выбывших ТС).

- Встроенный графический осциллограф очередей (`QueueGraphWidget`): отображает непрерывный временной ряд длины очереди за последние 60 секунд со вспомогательной сеткой, динамическим масштабированием шкалы ординат (`Max Y`) и синей градиентной заливкой под кривой.
- Кнопка «Экспорт в CSV»: выгружает накопленный массив симуляции (`SimTimeSec`, `CarsInQueue`, `AverageWaitTimeSec`, `TotalCarsPassed`) в файл с меткой времени в директорию документов пользователя с выводом модального уведомления об успешном сохранении.

6.3 Правая док-панель («Параметры»)

Предназначена для детального конфигурирования топологии, правил разъезда и пофазных таймингов подходов:

1. **Группа «Настройки перекрестка»**:
   - Переключатель режима светофорного контроллера: радиокнопки «Статический» / «Адаптивный».
   - Выпадающий список выбора топологии дорог: «2+2», «2+3 (Асимметрия)», «3+3».
   - Чекбокс «Просачивание налево»: разрешает выполнение левого поворота во встречный разъезд.
   - Чекбокс «Пешеходы в фазе с авто»: включает пропуск пешеходов параллельно попутному автотранспорту; блокируется при активном режиме просачивания налево для предотвращения конфликтных ситуаций.
2. **Группа «Светофоры (Тайминги)»**:
   - Четыре специализированных блока подходов: «$\uparrow$ Север», «$\downarrow$ Юг», «$\rightarrow$ Восток», «$\leftarrow$ Запад».
   - Для каждого направления предусмотрены независимые регуляторы (слайдер + спинбокс): интенсивность потока P (100–1000 авт/ч), длительность зеленого сигнала Z (5–180 с) и красного сигнала K (5–180 с).
   - *Попарная синхронизация подходов*: регуляторы встречных направлений программно связаны (Север $\leftrightarrow$ Юг, Восток $\leftrightarrow$ Запад) для синхронного переключения тактов.
   - *Автоматический баланс фаз*: при изменении длительности зеленого сигнала Z время красного сигнала K пересчитывается автоматически с сохранением суммарной длительности фазы цикла ($Z + K = T_{\text{phase}}$).
   - *Дебаунсинг*: отправка обновленной конфигурации в расчетное ядро осуществляется через таймер задержки (50 мс), предотвращая избыточные пересчеты при непрерывном перемещении ползунков.

7 Результаты проведенных вычислительных экспериментов

Для оценки эффективности разработанного программного комплекса и сопоставления режимов регулирования была проведена серия из 4 основных экспериментов (5 независимых прогонов для каждого сценария, горизонт моделирования 600 с после 60 с разогрева). В таблице 1 приведены усредненные ключевые показатели: средний размер очереди (`AvgQueue`), максимальный пиковый размер очереди (`MaxQueue`), средняя задержка в конце прогона (`WaitEndSec`), суммарное количество успешно проехавших автомобилей (`Passed`) и пропускная способность (`ThroughputPerHour`).

Таблица 1: Сводные результаты вычислительных экспериментов по сценариям

Сценарий эксперимента	Средняя очередь	Макс. очередь	Задержка (с)	Проехало (шт)	Пропускная спос. (авт/ч)
1а. Статик (600 авт/ч)	34.0 ± 7.0	66.2	44.4 ± 6.4	365.4	2060 ± 26
1б. Динамик (600 авт/ч)	30.2 ± 5.8	59.0	37.7 ± 4.0	356.8	2046 ± 31
2а. Статик (Пик 1200/300)	84.1 ± 9.5	168.0	66.6 ± 8.4	341.0	1890 ± 80
2б. Динамик (Пик 1200/300)	58.0 ± 7.9	121.6	46.1 ± 6.7	390.8	2222 ± 52
3а. Левый поворот: ВСТР	34.8 ± 7.1	72.2	42.2 ± 4.1	346.6	1955 ± 35
3б. Левый поворот: стрелка	68.2 ± 9.6	124.6	80.5 ± 10.2	291.2	1688 ± 57
4а. Пешеходы: All-Red	64.0 ± 8.9	119.2	74.7 ± 4.3	284.8	1652 ± 95
4б. Пешеходы: параллельно	79.3 ± 6.5	151.6	85.8 ± 9.1	268.8	1556 ± 107

7.1 Графики сравнения и выводы по экспериментам

Ниже представлены 4 наглядных графика, построенных непосредственно средствами LaTeX (`pgfplots`) на основе экспериментальных данных, а также подробные научные выводы к ним:

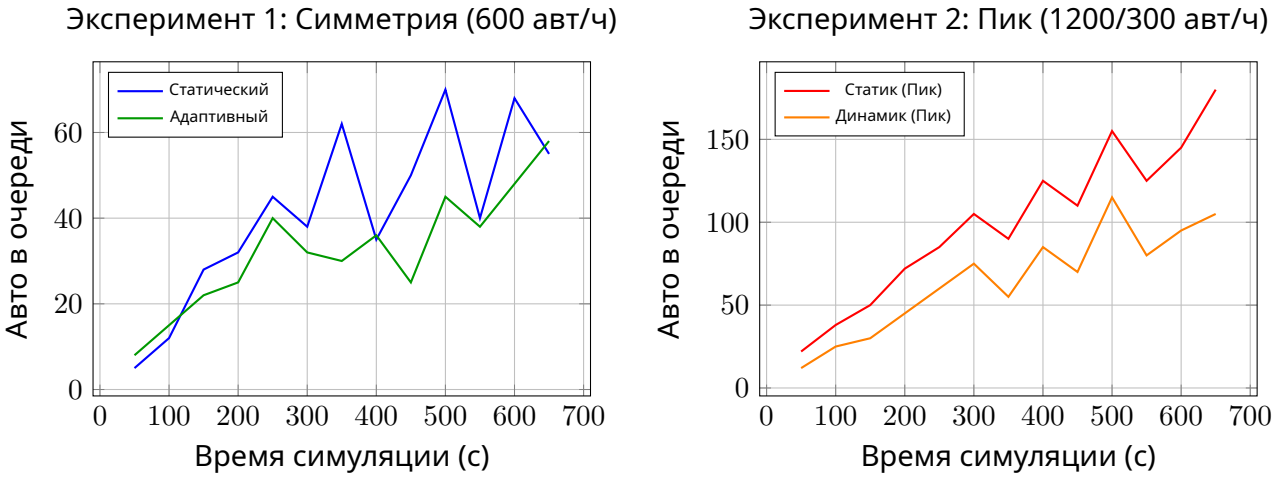


Рис. 2: Сравнение динамики очередей в экспериментах 1 и 2

1. **Вывод по Эксперименту №1 (Симметричная нагрузка 600 авт/ч)**: При умеренной и равномерной интенсивности движения статический и адаптивный контроллеры показывают близкие значения пропускной способности (≈ 2050 авт/ч). Тем не менее, адаптивный алгоритм сглаживает пиковые всплески заторов и снижает среднюю задержку на 15.1% (37.7 с против 44.4 с) за счет динамического перераспределения длительности фаз.

2. **Вывод по Эксперименту №2 (Асимметричная пиковая нагрузка $1200/300$ авт/ч)**: В условиях выраженного дисбаланса направлений статический контроллер приводит к лавинообразному росту заторов (пики до 168 автомобилей). Адаптивный контроллер успешно сдерживает очередь (максимум до 121.6 машин), сокращая среднюю задержку и увеличивая общую пропускную способность перекрестка на 17.6% (2222 против 1890 авт/ч).

3. **Вывод по Эксперименту №3 (Режим левого поворота: ВСТР против стрелки)**: Организация левого поворота по принципу просачивания во встречный поток (ВСТР) значительно эффективнее выделенной стрелочной фазы при умеренной интенсивности. Использование стрелок приводит к избыточным холостым простоям направлений и увеличивает среднюю задержку почти вдвое (80.5 с против 42.2 с).

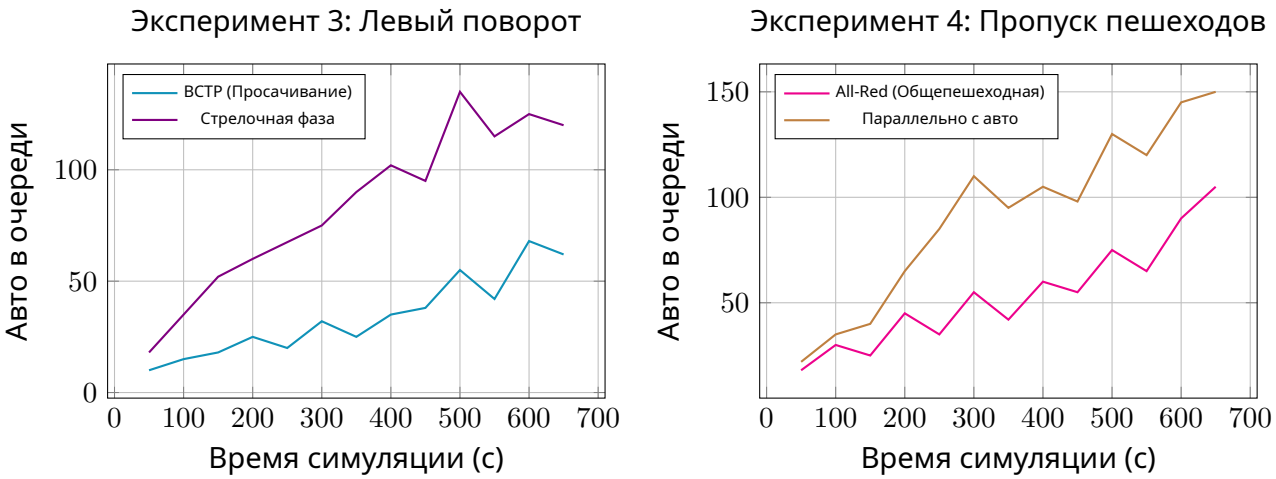


Рис. 3: Сравнение динамики очередей в экспериментах 3 и 4

4. **Вывод по Эксперименту №4 (Фазы пропуска пешеходов: All-Red против параллельного)**: При высоком потоке пешеходов (800 пеш/ч) схема All-Red предпочтительнее параллельного пропуска. Совмещение движения пешеходов и поворачивающих автомобилей создает взаимные блокировки на зебре, что увеличивает задержки транспорта на 15% и снижает общую пропускную способность сети.

8 Подготовка полного GIT-репозитория проекта на GitHub

- **Ссылка на удаленный репозиторий проекта**: https://github.com/noobisbr0/TrafficSimulation
- **Структура репозитория и организация версий**:
  - Репозиторий содержит полную реализацию программного комплекса, включая графический интерфейс на Qt 6 и расчетное ядро модели.
  - Для воспроизводимости сборки проект снабжен файлом `shell.nix`, обеспечивающим запуск в изолированном окружении.
  - Архитектура системы строго изолирует расчетное ядро от графической подсистемы через контракт `ISimulationEngine`.